



Assignment Title	Building and Securing a DevSecOps Pipeline
Learning outcomes covered	LO2: Apply secure software development practices including secure coding techniques, threat modelling and risk assessment methodologies to design secure software systems. LO3: Apply security automation frameworks, orchestration techniques and integration strategies to streamline and enhance security controls and processes within DevSecOps.
Assignment Mode	Group Project (4 members per group) with Group Technical Report and Individual Viva Voce
Maximum Marks	50 Marks (Technical Report: 25; Viva Voce: 25)
Contribution to the Final Grade	50% of the final module mark (Continuous Assessment)
Date published	To be announced
Deadline for submissions	To be announced
Mode of Submission	Courseweb: Technical Report (PDF), Ethical Clearance Form, and source-code repository link. Viva voce conducted as scheduled by the module team.

Assignment description

1. Overview and Rationale

This assignment asks each group to design, build and secure a small but genuinely working DevSecOps pipeline around an existing open-source application. Unlike a checklist exercise, the emphasis here is on demonstrating the DevSecOps mindset in practice: an attacker-identified vulnerability must be shown working before it is fixed, the CI/CD pipeline must be able to actually block a bad build, and secrets must be handled the way a real engineering team would handle them, not hardcoded and then "scanned for" afterwards.

The scope remains practical: everything can be run on a personal laptop using free and open-source tools, and no institutional cloud account, paid subscription, or Kubernetes cluster is required. Each group must select an open-source application that can be downloaded and run/tested completely offline, rather than building an application from scratch. The selected application must have at least two separate components that communicate with each other (for example: a frontend and an API/backend service, or an API/backend service and a database layer) **and must be containerisable with Docker.**

Suitable examples include:

- A simple open-source to-do list or task management app
- A mini open-source bookstore or e-commerce application
- An open-source notes-taking application
- A small open-source booking/reservation system
- Any other small open-source project (e.g. from GitHub/GitLab) that meets the two-component and containerization requirements above

Groups are not limited to the examples above and may propose any suitable open-source application, provided it meets the requirements and is confirmed with the team by the end of Week 1. **To reduce time spent searching, Appendix A provides a curated list of pre-vetted applications that meet the componentry, containerisation, and offline requirements; choosing from this list is optional but recommended.**

Depth of understanding is valued over scale: a small, genuinely well-secured application with a working pipeline that a group can defend confidently in the viva will score better than a large system with security bolted on superficially.

Appendix A, at the end of this document, is a curated list of pre-vetted open-source applications. A suggested 10-week schedule is also provided near the end of this document. Groups are encouraged to review both before finalizing their application choice in Week 1.

2. Project Tasks

2.1 Application, Architecture and Containerisation

- Select an application meeting the componentry requirement in Section 1 and containerise each component with its own Dockerfile (or use the project's existing Docker files if already containerized).
- Provide a docker-compose.yml that brings up the full application with one command.
- Draw an architecture diagram showing each component/service, the data flows between them, and the trust boundaries (e.g. public internet, internal network, database).

2.2 Threat Modelling and Risk Assessment

- Carry out a STRIDE-based threat model against the architecture diagram; identify at least 4 realistic, application-specific threats (generic threats copied from a template without justification will be marked down).
- Rate each threat's likelihood and impact (a simple 3x3 or 5x5 matrix is sufficient) and justify the rating in one or two sentences.
- For each threat, state which specific control in your system mitigates it, and where in the codebase or pipeline that control lives.

2.3 Secure Coding: Exploit-and-Fix

This is the core technical requirement of the assignment. For at least four vulnerabilities:

- Demonstrate the vulnerability working against the unmodified application (e.g. a SQL injection payload returning unauthorised data, a stored XSS payload executing, or an authentication bypass) and capture evidence (screenshots).
- Apply a secure coding fix (e.g. parameterised queries, output encoding, proper session/password handling, input validation, correct error handling).
- Re-attempt the exact same exploit against the fixed code and show it no longer succeeds.
- Run a free SAST tool (Semgrep or Bandit, or an equivalent appropriate to your stack) before and after the fix and include the before/after finding count or diff.

Vulnerability fixes that are not tied to a demonstrated exploit (i.e. "we improved input validation" with no proof it mattered) will be treated as basic secure coding evidence only, not as full vulnerability remediation.

2.4 CI/CD Pipeline and Security Automation

- Set up a CI/CD pipeline using GitHub Actions (or an equivalent free CI tool) that builds and tests your application automatically on every push.
- The pipeline must include all four of the following automated security gates:
 - SAST (Semgrep, Bandit, or ESLint security plugin, as appropriate to your language)
 - Dependency / software composition scanning (pip-audit, npm audit, or equivalent)
 - Secrets scanning (Gitleaks or equivalent)
 - Container image scanning (Trivy) against your built Docker image(s)
- A DAST scan with OWASP ZAP against the running application is an optional addition for extra credit under CI/CD Pipeline & Security Automation.
- At least one gate must be configured to genuinely fail the pipeline (not just warn) when it detects a real issue above an agreed severity threshold; capture a pipeline run where this happens, either on a real finding or one deliberately reintroduced for demonstration.

2.5 Secrets Management

- No credentials, API keys, or connection strings may be hardcoded or committed to the repository at any point in the final submission's history that is presented as current practice.
- Use GitHub Actions encrypted secrets at minimum; a group that additionally demonstrates HashiCorp Vault (or another free secrets manager) injecting a secret at runtime will be marked more favourably under CI/CD criterion.
- Briefly document, in the report, how secrets are provisioned to the running application and to the pipeline.

3. Submission Components

Each group must submit all of the following as a single, complete package. Incomplete submissions will be marked based only on what is verifiable.

3.1 Ethical Clearance Form

- A signed Ethical Clearance Form confirming the selected open-source application and the group's planned activities (including any exploit demonstration) comply with the Institute's ethical and legal guidelines.
- Must be submitted alongside the other components; work carried out without a submitted form may not be accepted.

3.2 Source Code Repository

- A GitHub (or equivalent free) repository containing all application code, Dockerfiles/docker-compose.yml, CI/CD pipeline configuration, and a README with clear setup/run instructions.
- Commit history should reflect contributions from all 4 members.
- Repository link (with access granted to the module team) must be included in the report.

3.3 Technical Report (PDF, 1800–2500 words excluding appendices)

- Table of contents.
- Executive summary and system overview (architecture diagram, tech stack, containerisation approach, and the open-source application selected).
- Threat model, risk assessment and threat-to-control mapping (LO2).
- Secure coding evidence: exploit-and-fix walkthrough with before/after SAST results (LO2).
- CI/CD pipeline design, all four security gates, and evidence of at least one gate blocking a build (LO3).
- Secrets management approach.
- Short reflection: what you would improve with more time.
- Individual contribution statement signed off by all group members, including disclosure of any AI tool usage.
- References in a consistent citation style (e.g. IEEE).

3.4 Viva Voce (Group)

- Each group attends a viva of approximately 10 minutes.

Submission Guidelines

- All components must be submitted via Courseweb (<http://courseweb.sliit.lk>) by the published deadline; late submissions are subject to the Faculty's standard late-penalty policy.
- Only one submission per group is required; all members must be listed with their student IDs and assigned role.
- The signed Ethical Clearance Form must be included in the submission package.

Academic Integrity

Any type of plagiarism is not allowed. Academic honesty is crucial to a student's credibility and self-esteem and ultimately reflects the values and morals of the Institute as a whole. Groups may discuss general concepts with other groups, but all submitted code, diagrams, report text must be the group's own original work; any external code, tools or reference material must be clearly cited.

Usage of AI: Groups may use AI tools (e.g. ChatGPT, GitHub Copilot, Claude) to support coding, debugging, or drafting, provided such use is clearly and honestly disclosed in the report's Individual Contribution Statement, stating which tool(s) were used, for what purpose, and for which parts of the submission. Undisclosed or inappropriate use of AI, including wholesale AI generation of the report or code without disclosure and AI-assisted viva answers is strictly prohibited, is treated as an academic integrity violation.

Suggested 10-Week Schedule

This schedule is a recommended pacing guide, not a set of separately marked milestones. Groups that fall behind it typically run out of time for the CI/CD and secret management work, which carries the most marks.

Week	Focus	Checkpoint
1	Application selection; confirm with module team; repo + Docker Compose skeleton set up	App confirmed; both components run locally via Docker
2	Architecture diagram; data flow mapping; STRIDE threat modelling	Draft architecture diagram and threat list (4+ threats)
3	Risk assessment (likelihood/impact); map each threat to a planned control	Threat-to-control mapping table drafted
4	Vulnerability proof-of-concept: demonstrate at least 4 exploitable weaknesses in the unmodified app	Screenshots/recording of each exploit working
5	Secure coding fixes for the demonstrated vulnerabilities; re-test to confirm the exploit no longer works	Before/after SAST diff; exploit re-attempted and blocked
6	GitHub Actions pipeline: build + automated tests on every push	Pipeline runs green on a clean commit
7	Add SAST, dependency scanning and secrets scanning gates; configure fail conditions	Pipeline fails on a deliberately reintroduced issue
8	Container image scanning (Trivy); secrets management (no hardcoded credentials)	Image scan integrated; secrets pulled from GitHub Actions secrets/Vault, not source
9	Buffer week: optional DAST/IaC scanning; polish evidence; draft report	Full pipeline run captured end-to-end
10	Report finalisation, video recording, viva preparation	All submission components ready

Marking Scheme

Technical Report Assessment Rubric (25 marks)

Assessment Criterion	Marks	Excellent (85–100%)	Good (65–84%)	Satisfactory (50–64%)	Unsatisfactory (<50%)
Executive Summary & System Overview	3	Clear, concise and comprehensive overview; selected open-source application, architecture and technology stack are presented effectively.	Good overview with appropriate architecture and technology details; minor omissions.	Basic overview provided with limited architectural or technical detail.	Unclear, incomplete or missing system overview.
Threat Modelling & Risk Assessment (LO2)	5	Comprehensive threat model with realistic threats, appropriate risk assessment and well-justified mitigations.	Good threat model and risk assessment with minor gaps in analysis or mitigation mapping.	Basic threat model covering major threats with limited risk analysis.	Threat model is incomplete, inaccurate or poorly justified.
Vulnerability Assessment & Secure Coding Evidence (LO2)	6	Vulnerabilities are clearly identified and analysed; at least two fixes are well implemented with strong before/after	Appropriate vulnerabilities identified and fixes implemented with good supporting evidence; minor gaps present.	Basic vulnerabilities and fixes presented with limited analysis or validation evidence.	Vulnerabilities are poorly identified, fixes are inappropriate/incomplete, or evidence is missing.

		and re-scan evidence.			
CI/CD Pipeline Design & Security Automation (LO3)	5	Well-designed pipeline with effective automated security checks, clear workflow and strong scan/execution evidence.	Functional pipeline with appropriate security automation and minor implementation/documentation gaps.	Basic pipeline implemented with limited security automation or supporting evidence.	Pipeline is incomplete/non-functional or meaningful security automation is absent.
Industry Trends & Case Study Analysis	3	Strong analysis of relevant industry trends/case study with clear connections to DevSecOps and project security practices.	Relevant analysis with good DevSecOps connections but limited depth in some areas.	Basic discussion of industry trends/case study with limited critical analysis.	Minimal, irrelevant or missing industry analysis.
Reflection & Future Improvements	1	Insightful reflection with realistic and technically relevant improvements.	Good reflection with appropriate improvements.	Basic reflection with limited justification.	Superficial or missing reflection.
Contribution Statement & AI Usage Disclosure	1	Contributions are clearly documented and AI usage is transparent	Mostly complete with minor omissions.	Basic contribution information provided.	Missing or unclear contribution/AI disclosure.

		y disclosed where applicable.			
References & Overall Report Quality	1	Professional presentation with consistent citations and referencing throughout.	Well presented with minor referencing/formatting issues.	Acceptable presentation with several inconsistencies.	Poor presentation or inadequate referencing.
Total	25				

Viva Assessment Rubric (25 marks)

Assessment Criterion	Marks	Excellent (85–100%)	Good (65–84%)	Satisfactory (50–64%)	Unsatisfactory (<50%)
Understanding of the Open-Source Application & Architecture	3	Demonstrates comprehensive understanding of the selected application, architecture, technologies and data flow.	Good understanding with minor gaps or inaccuracies.	Demonstrates basic understanding of the application and architecture.	Unable to adequately explain the application or architecture.
Threat Modelling & Risk Assessment	4	Confidently explains identified threats, risk ratings, attack scenarios and mitigation decisions with strong justification.	Good understanding of threats and mitigations with minor gaps.	Basic understanding demonstrated but requires prompting.	Unable to explain or justify the threat model and risks.
Vulnerability Assessment & Secure Coding Fixes	5	Clearly explains identified vulnerabilities, their security impact, vulnerable code and	Good understanding of vulnerabilities and fixes with	Can explain basic vulnerabilities and fixes but lacks technical depth.	Unable to explain vulnerabilities or justify implemented fixes.

		implemented fixes; demonstrates strong secure coding knowledge.	minor technical gaps.		
CI/CD Pipeline & Security Automation	5	Confidently explains and demonstrates the pipeline workflow, security tools, automated scans, results and security gates.	Good explanation of pipeline and security automation with minor gaps.	Basic understanding of pipeline execution and security tools.	Unable to explain how the pipeline or security automation works.
Industry Trends & Case Study Understanding	2	Demonstrates strong understanding of the selected industry case/trend and clearly relates lessons learned to DevSecOps practices.	Good understanding with appropriate connections to DevSecOps.	Basic understanding with limited connection to project concepts.	Unable to explain the selected case/trend or its relevance.
Technical Knowledge & Problem-Solving	4	Answers technical and scenario-based questions accurately, independently and with strong justification.	Answers most questions correctly with reasonable justification.	Answers basic questions but requires prompting and demonstrates limited reasoning.	Unable to answer fundamental technical or scenario-based questions.
Communication & Professional Skills	2	Communicates confidently, clearly and professionally; explanations are logical and well structured.	Communicates clearly with minor hesitation.	Adequate communication but lacks confidence or clarity.	Poor communication or unable to articulate technical responses effectively.
Total	25				

Appendix A: Curated List of Approved Open-Source Applications

The applications below are pre-vetted against the requirements in Section 1: at least two communicating components, fully offline operation, free/open-source dependencies throughout, and straightforward containerisation. Groups may still propose an application not on this list, subject to confirmation with the module team by the end of Week 1.

The first group (intentionally vulnerable applications) is recommended for groups that want to focus their limited time on the pipeline and remediation work rather than on vulnerability discovery: the flaws are documented, realistic, and mapped to known CWE/OWASP categories, which makes the Section 2.3 exploit-and-fix requirement far more predictable to plan and demonstrate.

A.1 Intentionally Vulnerable Applications (recommended)

Application	Stack	Components	Docker Readiness	Notes
OWASP NodeGoat	Node.js / Express / MongoDB	Web app + database	Official Dockerfile provided	Vulnerabilities mapped to OWASP Top 10; strong fit for STRIDE threat modelling
OWASP Juice Shop	Node.js / Angular	Frontend + backend API	Official Docker image (bkimminich/juice-shop)	Very well documented; large community write-ups can help groups but must not be copied into the report
DVWA (Damn Vulnerable Web App)	PHP / MySQL	Web app + database	Official image (vulnerables/web-dvwa)	Adjustable difficulty levels; good for showing a graduated before/after fix
OWASP WebGoat	Java / Spring Boot	Web app + embedded database	Official Docker image (webgoat/webgoat)	Best fit for Java-inclined groups; lessons map cleanly to secure coding fixes
Mutillidae II	PHP / MySQL	Web app + database	Docker support via webpwnized/mutillidae	Broad vulnerability catalogue; useful if a group wants extra options for Section 2.3
OWASP crAPI (Completely Ridiculous API)	Node.js / Go / Python microservices + MongoDB + PostgreSQL	Multiple API services + two databases	Official docker-compose.yml provided (full stack)	API-specific vulnerabilities (BOLA, mass assignment, SSRF); excellent fit for a DevOps Security module, but has more

				moving parts than the others, recommend only for groups comfortable managing several services at once
--	--	--	--	---

A.2 Standard Applications (find/introduce your own issues)

These are ordinary small open-source projects with no deliberately planted vulnerabilities. Groups choosing from this group should expect to spend more time in Weeks 3–4 identifying genuine weaknesses (often in older dependency versions or missing input validation) and should budget schedule time accordingly.

Application Type	Typical Stack	Components	Docker Readiness	Notes
Node/Express + MongoDB to-do or task manager	Node.js / Express / MongoDB	API + database	Usually needs a Dockerfile written; MongoDB has an official image	Search GitHub for "node express mongodb todo docker-compose"
Flask or Django note-taking app	Python / Flask or Django / PostgreSQL	API + database	Usually needs a Dockerfile written; Postgres has an official image	Search GitHub for "flask notes app postgres" or "django todo docker"
Spring Boot + MySQL booking/reservation system	Java / Spring Boot / MySQL	API + database	Often ships a Dockerfile already	Search GitHub for "spring boot booking system mysql"
MERN-stack bookstore/e-commerce app	MongoDB / Express / React / Node.js	Frontend + backend API + database	Varies; check repo before selecting	Search GitHub for "mern bookstore" or "mern ecommerce docker"
RealWorld / Conduit (Medium.com clone)	Any 100+ community implementations across Node, Django, Spring, Rails, Go, .NET, etc.	Frontend SPA + backend API (+ database in most backends)	Varies by implementation; many pairs already include a Dockerfile	A single specification implemented consistently across many stacks (see github.com/gothinkster/realworld); good choice if the group wants a clean, well-structured codebase and a language they already know, since it is not vulnerable by design



Before selecting a Standard Application, confirm it satisfies the four-point check from the module's supplementary guidance: (1) a Dockerfile or docker-compose.yml already exists or is straightforward to write, (2) it is a web app/API/CLI service, not a desktop GUI or mobile app, (3) it runs fully offline with no live third-party cloud dependency, and (4) every dependency, including the base OS image, is free and open-source.